

Tag 8

Dateirechte und sudo

Rechte lesen und mit `chmod` ändern, mit Buchstaben und mit Zahlen. Besitzer ändern mit `chown`. Einen Ordner für eine Gruppe bauen und Permission denied verstehen.

Warum das wichtig ist

Gestern hast du dieses Beispiel gesehen: Du wechselst mit `su javier` zu Javier, ohne Bindestrich. Javier bleibt dann in deinem Home-Ordner. Er tippt `ls`. Linux sagt:

```
ls: cannot open directory '.': Permission denied
```

Warum darf Javier das nicht? Linux schreibt an jede Datei und an jeden Ordner, wer was damit tun darf. Das sind die **Rechte**. Schau dir die Rechte deines Home-Ordners an:

```
ls -ld ~  
# drwxr-x--- 15 carlos carlos 4096 Sep 30 08:10 /home/carlos
```

Vorne stehen zehn Zeichen: `drwxr-x---`. Am Ende von Teil 1 kannst du sie lesen. Nur so viel schon jetzt: Die letzten drei Striche heißen „alle anderen dürfen nichts“. Javier gehört zu „alle anderen“. Darum kommt der Fehler.

Rechte schützen deine Dateien vor anderen Benutzern. Sie schützen auch das System. Das hast du gestern gesehen: Ohne `sudo` durftest du keinen Benutzer anlegen. Heute lernst du, Rechte zu lesen und zu ändern.

Eine Zeile von `ls -l` lesen

`ls -l` kennst du von Tag 3. Heute schauen wir genau hin. Die Datei mit den Passwörtern ist ein gutes Beispiel:

```
ls -l /etc/shadow  
# -rw-r----- 1 root shadow 1420 Sep 29 14:40 /etc/shadow
```

Teil	Bedeutung
-	die Art. - ist eine Datei, d ein Ordner
rw-	was der Besitzer darf
r--	was die Gruppe darf
---	was alle anderen dürfen
root	der Besitzer. Meist hat er die Datei angelegt
shadow	die Gruppe der Datei

Die 1, die Größe und das Datum brauchst du heute nicht.

Bei deinen eigenen Dateien steht zweimal dein Name, zum Beispiel `carlos carlos`. Warum? Ubuntu gibt jedem Benutzer eine eigene Gruppe mit seinem Namen. Das hast du gestern bei `id` gesehen: `gid=1000(carlos)`.

Für einen Ordner nimmst du `ls -ld`. Das `-d` zeigt den Ordner selbst. Ohne `-d` zeigt `ls -l`, was im Ordner liegt.

Welcher Block gilt für dich? Linux fragt der Reihe nach. Bist du der Besitzer? Dann gilt der erste Block. Sonst: Bist du in der Gruppe der Datei? Dann gilt der zweite. Sonst gilt der dritte. Es gilt immer nur ein Block.

r, w und x

In jedem Block gibt es drei Plätze. Ein Buchstabe heißt: erlaubt. Ein - heißt: verboten.

Recht	bei einer Datei	bei einem Ordner
r read	lesen, zum Beispiel mit <code>cat</code>	die Namen darin zeigen mit <code>ls</code>
w write	ändern	Dateien darin anlegen und löschen
x execute	als Programm starten	hineingehen mit <code>cd</code>

Das `x` kennst du von Tag 4. Ohne `x` startet kein Skript.

Jetzt kannst du deinen Home-Ordner lesen: `drwxr-x---`

- `d` : ein Ordner.
- `rw` : Carlos darf alles.
- `r-x` : die Gruppe `carlos` darf hinein und die Namen sehen.
- `---` : alle anderen dürfen nichts. Auch Javier nicht.

Ein Ordner braucht fast immer `r` und `x` zusammen. Ohne `x` kommst du nicht hinein. Ohne `r` siehst du keine Namen.

chmod mit Buchstaben

`chmod` ändert die Rechte. Der Name kommt von „change mode“. Das darf nur der Besitzer. Du sagst drei Dinge: für wen, dazu oder weg, welches Recht.

Buchstabe	Englisch	für wen
u	user	den Besitzer
g	group	die Gruppe
o	others	alle anderen
a	all	alle drei

+ gibt ein Recht dazu. - nimmt es weg.

Zum Ausprobieren legst du einen Ordner und zwei leere Dateien an:

```
mkdir -p ~/linux-kurs/tag08
cd ~/linux-kurs/tag08
touch plan.txt test.sh
```

Vorher und nachher:

```
ls -l plan.txt
# -rw-rw-r-- 1 carlos carlos 0 Sep 30 09:20 plan.txt
chmod g-w plan.txt
chmod o-r plan.txt
```

```
ls -l plan.txt
# -rw-r----- 1 carlos carlos 0 Sep 30 09:20 plan.txt
```

g-w : Die Gruppe darf die Datei nicht mehr ändern. o-r : Alle anderen dürfen sie nicht mehr lesen. So darf nur der Besitzer test.sh starten: `chmod u+x test.sh`

`chmod +x` von Tag 4 hat kein u, g oder o davor. Dann gilt es für alle drei.

chmod mit Zahlen

Jedes Recht hat eine Zahl: r ist 4, w ist 2, x ist 1. Pro Block zählst du zusammen.

Zahl	Rechte	so gerechnet
7	rwX	4 + 2 + 1
6	rw-	4 + 2
5	r-X	4 + 1
4	r--	4
0	---	nichts

Drei Blöcke geben drei Ziffern: Besitzer, Gruppe, andere. Eine neue Datei hat `rw-rw-r--`. Das ist 6, 6 und 4, also **664**.

Diese fünf Zahlen brauchst du fast immer:

Zahl	Rechte	wofür
644	rw-r--r--	eine Datei, die alle lesen dürfen
600	rw-----	eine geheime Datei, nur für dich
755	rxrx-rx-rx	ein Skript oder Ordner für alle
700	rwX-----	ein Skript oder Ordner nur für dich
770	rxwxrwX---	ein Ordner für eine Gruppe. Den baust du in Teil 2

```
chmod 600 plan.txt
ls -l plan.txt
# -rw----- 1 carlos carlos 0 Sep 30 09:25 plan.txt
```

Buchstaben ändern ein einzelnes Recht. Zahlen setzen alle neun auf einmal. Beides ist richtig. In Skripten und in der Klausur siehst du meist Zahlen.

Achtung: `chmod 777` erlaubt jedem alles. Das ist fast nie richtig. Suche lieber, welches Recht genau fehlt.

Häufige Fehler

- **Operation not permitted bei `chmod`.** Die Datei gehört dir nicht. Nur der Besitzer darf die Rechte ändern. Prüfe mit `ls -l`, wem sie gehört.
- **Nur zwei Ziffern.** `chmod 64 datei.txt` heißt `064`. Dann darfst du selbst nichts mehr: `----rw-r--`. Schreib immer drei Ziffern.
- **o verwechselt.** `o` heißt others, also alle anderen. Der Besitzer ist `u`.
- `ls -l ordner` **zeigt die Rechte des Ordners nicht.** Es zeigt, was darin liegt. Nimm `ls -ld ordner`.
- **Permission denied bei `./skript.sh`.** Das `x` fehlt. Tippe `chmod u+x skript.sh`.
- **cd geht nicht, obwohl `r` da ist.** Bei einem Ordner heißt `x` hineingehen. Gib `x` dazu.

Aufgaben · Teil 1

Rechte lesen

Wozu? Bevor du Rechte änderst, musst du sie lesen können. Hier liest du die Rechte deiner Dateien und von zwei wichtigen Dateien des Systems.

1. Lege den Ordner für heute an: `mkdir -p ~/linux-kurs/tag08`
2. Wechsle hinein: `cd ~/linux-kurs/tag08`
3. Lege eine Datei an. Tippe im Terminal: `echo "Milch und Brot" > liste.txt`
4. Zeige ihre Rechte: `ls -l liste.txt`
5. Lies die ersten zehn Zeichen: `-rw-rw-r--` . Du und deine Gruppe dürft lesen und ändern. Alle anderen dürfen nur lesen.
6. Zeige die Rechte deines Home-Ordners: `ls -ld ~`
7. Zeige die Rechte der Benutzerliste: `ls -l /etc/passwd` . Alle dürfen sie lesen. Nur root darf sie ändern. Darum brauchte `useradd` gestern `sudo` .
8. Zeige die Rechte der Datei mit den Passwörtern: `ls -l /etc/shadow` . Alle anderen haben `--` .
9. Versuche, sie zu lesen: `cat /etc/shadow` . Es kommt `Permission denied` .

So sieht es aus. Statt `carlos` steht dort dein Name. Größe und Datum sind bei dir anders:

```
-rw-rw-r-- 1 carlos carlos 15 Sep 30 09:12 liste.txt
drwxr-x--- 15 carlos carlos 4096 Sep 30 08:10 /home/carlos
-rw-r--r-- 1 root root 2847 Sep 21 10:02 /etc/passwd
-rw-r----- 1 root shadow 1420 Sep 29 14:40 /etc/shadow
cat: /etc/shadow: Permission denied
```

✓ Bestätigt

Rechte mit Buchstaben ändern

Wozu? Oft fehlt nur ein einziges Recht. Mit Buchstaben gibst oder nimmst du genau dieses eine. Der Rest bleibt, wie er ist.

1. Wechsle in den Ordner von heute: `cd ~/linux-kurs/tag08`
2. Lege ein Tagebuch an: `echo "Mein Tagebuch" > tagebuch.txt`
3. Zeige die Rechte: `ls -l tagebuch.txt`. Du siehst `-rw-rw-r--`.
4. Nimm allen anderen das Lesen: `chmod o-r tagebuch.txt`
5. Nimm der Gruppe das Schreiben: `chmod g-w tagebuch.txt`
6. Prüfe: `ls -l tagebuch.txt`. Jetzt steht dort `-rw-r-----`.
7. Öffne eine neue Datei: `nano gruss.sh`
8. Tippe diese 2 Zeilen:

```
#!/bin/bash
echo "Hallo aus Tag 8"
```

9. Speichere mit Strg+O und Enter. Beende mit Strg+X.
10. Starte das Skript: `./gruss.sh`. Es kommt `Permission denied`. Das `x` fehlt.
11. Gib dir das Recht zum Starten: `chmod u+x gruss.sh`
12. Starte es noch einmal: `./gruss.sh`. Jetzt klappt es.
13. Prüfe: `ls -l gruss.sh`. Beim Besitzer steht jetzt `rw`.

So sieht es aus. Statt `carlos` steht dort dein Name. Größe und Datum sind bei dir anders:

```
-rw-r----- 1 carlos carlos 14 Sep 30 09:20 tagebuch.txt
bash: ./gruss.sh: Permission denied
Hallo aus Tag 8
-rwxrw-r-- 1 carlos carlos 35 Sep 30 09:22 gruss.sh
```

✓ Bestätigt

Drei Dateien, drei Zahlen

Wozu? Mit Zahlen setzt du alle Rechte auf einmal. So steht es oft in Skripten und in der Klausur.

1. Wechsle in den Ordner von heute. Tippe im Terminal: `cd ~/linux-kurs/tag08`
2. Lege drei leere Dateien an: `touch geheim.txt oeffentlich.txt start.sh`
3. Nur du darfst lesen und schreiben: `chmod 600 geheim.txt`
4. Alle dürfen lesen, nur du schreibst: `chmod 644 oeffentlich.txt`
5. Alle dürfen starten, nur du änderst: `chmod 755 start.sh`
6. Prüfe alle drei: `ls -l geheim.txt oeffentlich.txt start.sh`
7. Rechne eine Zeile nach. `rw-r--r--` ist 4 + 2, dann 4, dann 4. Also `644` .

So sieht Schritt 6 aus:

```
-rw----- 1 carlos carlos 0 Sep 30 09:25 geheim.txt
-rw-r--r-- 1 carlos carlos 0 Sep 30 09:25 oeffentlich.txt
-rwxr-xr-x 1 carlos carlos 0 Sep 30 09:25 start.sh
```

✓ **Bestätigt**

Quiz · Teil 1

Das Quiz gibt keine Punkte. Du kannst es so oft wiederholen, wie du willst.

Quiz abgegeben. Ergebnis: 5 / 5 richtig

**1. `ls -l` zeigt: `-rw-r----- 1 carlos verkauf 300 Sep 30 10:00 bericht.txt` .
Javier ist in der Gruppe verkauf . Was darf Javier mit der Datei?**

Nur lesen. (Deine Antwort)

Lesen und ändern.

Nichts.

Lesen, ändern und starten.

Richtig

Erklärung:

Javier ist nicht der Besitzer, aber in der Gruppe. Für ihn gilt der mittlere Block `r--` . Er darf nur lesen.

2. Welche Zahl passt zu `rwxr-xr-x` ?

744

755 (Deine Antwort)

775

655

Richtig

Erklärung:

`rw` ist $4 + 2 + 1 = 7$. `r-x` ist $4 + 1 = 5$. Zweimal `r-x` gibt 5 und 5. Also 755 .

3. Du tippst `chmod o-r tagebuch.txt`. Wer darf danach nicht mehr lesen?

Der Besitzer.

Die Gruppe.

Alle anderen. (Deine Antwort)

Niemand mehr.

Richtig

Erklärung:

`o` heißt others, also alle anderen. Der Besitzer ist `u`, die Gruppe ist `g`.

4. Bei einem Ordner hast du `r`, aber kein `x`. Was passiert bei `cd` in diesen Ordner?

`cd` klappt, aber `ls` nicht.

Permission denied. (Deine Antwort)

`cd` klappt ganz normal.

`cd` klappt, aber du siehst keine Namen.

Richtig

Erklärung:

Bei einem Ordner heißt `x` hineingehen. Ohne `x` gibt `cd` `Permission denied`, auch mit `r`.

5. Nur der Besitzer von `start.sh` soll das Recht zum Starten bekommen. Schreib den ganzen Befehl mit Buchstaben.

Deine Antwort: `chmod u+x start.sh`

Richtig

Erklärung:

u ist der Besitzer, + gibt dazu, x ist Starten: `chmod u+x start.sh`

Bonusaufgaben · Teil 1

Freiwillig, für alle, die früher fertig sind. Bonusaufgaben zählen nicht für die Tagesmedaille. Sind alle Bonusaufgaben eines Tages bestätigt, gibt es eine Bonusmedaille.

Rechte-Rätsel Bonus

Wozu? Wer Rechte schnell lesen kann, findet Fehler schneller. Hier liest du sechs Zeilen ohne Hilfe.

Das sind sechs Zeilen von `ls -l`:

```
-rw-r--r-- 1 carlos carlos 120 Sep 30 10:00 notizen.txt
-rwx----- 1 carlos carlos 80 Sep 30 10:00 backup.sh
drwxr-x--- 2 carlos verkauf 4096 Sep 30 10:00 angebote
-rw-rw---- 1 javier verkauf 300 Sep 30 10:00 preise.txt
-r----- 1 carlos carlos 40 Sep 30 10:00 pin.txt
drwxrwxr-x 3 carlos carlos 4096 Sep 30 10:00 projekt
```

1. Lege in `~/linux-kurs/tag08` mit `nano` die Datei `raetsel.txt` an.
2. Schreib für jede der sechs Zeilen eine Zeile in die Datei. Beantworte darin drei Dinge:
 - Ist es eine Datei oder ein Ordner?
 - Welche Zahl haben die Rechte?
 - Was darf der Besitzer, was die Gruppe, was alle anderen?
3. Beantworte darunter noch drei Fragen:
 - Javier ist in der Gruppe `verkauf`. Darf er in den Ordner `angebote` ?
 - Pablo ist in keiner dieser Gruppen. Darf er `notizen.txt` lesen?
 - Carlos will `pin.txt` ändern. Geht das? Was kann er tun?
4. Prüfe dich selbst an zwei Zeilen. Lege eine leere Datei an. Gib ihr die Zahl, die du ausgerechnet hast. Vergleiche mit `ls -l`.
5. Lösche die Testdateien wieder mit `rm`. Fragt `rm` nach `write-protected`, tippe `y` und Enter.

So kann eine Zeile in deiner Datei aussehen:

```
notizen.txt: Datei, 644. Besitzer lesen und ändern. Gruppe lesen. Andere lesen.
```

✓ Bestätigt

chmod-Training Bonus

Wozu? Übung macht sicher. Hier setzt du zwölf Mal Rechte. Die Befehle findest du selbst.

1. Lege in `~/linux-kurs/tag08` einen Ordner `training` an. Geh hinein.
2. Lege darin fünf leere Dateien an: `a.txt`, `b.txt`, `c.sh`, `d.txt`, `e.sh`. Lege dazu einen Ordner `f` an.
3. Bring alle sechs auf diese Rechte. Nimm dafür nur **Zahlen**:

Name	Ziel
<code>a.txt</code>	<code>-rw-----</code>
<code>b.txt</code>	<code>-rw-r--r--</code>
<code>c.sh</code>	<code>-rwxr-x---</code>
<code>d.txt</code>	<code>-r--r--r--</code>
<code>e.sh</code>	<code>-rwxrwxr-x</code>
<code>f</code>	<code>drwx-----</code>

4. Prüfe mit `ls -l`. Stimmen alle sechs?
5. Setze alles zurück. Tippe dafür diese 2 Zeilen:

```
chmod 664 a.txt b.txt c.sh d.txt e.sh
chmod 775 f
```

6. Bring alle sechs noch einmal auf dieselben Ziele. Diesmal nur mit **Buchstaben**: `u`, `g`, `o`, `a`, `+` und `-`.
7. Prüfe wieder mit `ls -l`.
8. Räume auf. Geh zurück in `~/linux-kurs/tag08` und lösche den Ordner: `rm -r training`

Tipps:

- Mit Buchstaben brauchst du für manche Dateien zwei oder mehr Befehle.
- Du darfst mehrere Rechte hinter ein Zeichen schreiben, zum Beispiel `g-rw`.
- Fragt `rm` in Schritt 8 nach `write-protected`, tippe `y` und Enter. `d.txt` hat kein `w`.

✓ Bestätigt

Ein Versuch mit einem Ordner Bonus

Wozu? Bei einem Ordner heißen `r` und `x` etwas anderes als bei einer Datei. Das merkst du dir am besten, wenn du es selbst ausprobierst. Erst rätst du, dann testest du.

1. Lege in `~/linux-kurs/tag08` einen Ordner `versuch` an.
2. Lege darin die Datei `datei.txt` mit dem Text `hallo` an.
3. Lege mit `nano` die Datei `vermutung.txt` an. Schreib hinein, was du glaubst: Was machen diese drei Befehle, wenn dem Ordner das `r` fehlt? Und was, wenn ihm das `x` fehlt?

```
ls versuch
cd versuch
cat versuch/datei.txt
```

4. Nimm dem Besitzer beim Ordner `versuch` das `r` weg. Den Befehl findest du selbst.
5. Teste die drei Befehle. Klappt `cd`, geh mit `cd ..` zurück.
6. Gib das `r` zurück. Nimm dann das `x` weg. Teste wieder alle drei.
7. Schreib unter deine Vermutung, was wirklich passiert ist. Wo lagst du richtig?
8. Stell den Ordner wieder her: `chmod 775 versuch`
9. Räume auf: `rm -r versuch`

Tipp: Du bist der Besitzer. Für dich gilt nur der erste Block. Die Gruppe hilft dir hier nicht.

✓ Bestätigt

Das kaputte Projekt Bonus

Wozu? Ein falscher `chmod` kann einen ganzen Ordner unbrauchbar machen. Hier reparierst du so einen Ordner. Du findest selbst heraus, was fehlt.

1. Wechsle in den Ordner von heute: `cd ~/linux-kurs/tag08`
2. Baue das kaputte Projekt. Tippe diese 5 Zeilen:

```
mkdir -p projekt/skripte projekt/daten
echo "Kunde Meier" > projekt/daten/kunden.txt
echo "echo Backup laeuft" > projekt/skripte/backup.sh
chmod 644 projekt/skripte projekt/daten
chmod 644 projekt
```

Die letzten zwei Zeilen sind der Fehler. Jemand hat allen drei Ordnern die Rechte `644` gegeben.

3. Probiere, was noch geht: `cd projekt`, `ls projekt/daten`, `cat projekt/daten/kunden.txt`
4. Finde heraus, warum nichts mehr geht. Schau mit `ls -ld projekt`.
5. Repariere das Projekt. Am Ende soll es so aussehen:

Was	Ziel
projekt , projekt/skripte , projekt/daten	drwxr-xr-x
kunden.txt	-rw-----
backup.sh	-rwxr-xr-x

6. Prüfe mit `ls -ld projekt projekt/skripte projekt/daten` und mit `ls -l projekt/daten projekt/skripte`.
7. Starte das Skript: `./projekt/skripte/backup.sh`. Es zeigt `Backup laeuft`.
8. Räume auf: `rm -r projekt`

Tipps:

- Fang außen an. Warum? Das findest du beim Probieren heraus.
- Schreib dir am Ende in einem Satz auf: Warum war `644` für die Ordner so schlimm?

✓ Bestätigt

Ein Ordner für die ganze Gruppe

Gestern hast du die Gruppe `verkauf` angelegt und Javier hineingenommen. Die Idee war: Alle im Verkauf öffnen dieselben Dateien. Dafür brauchen sie einen gemeinsamen Ordner. Dein Home-Ordner geht dafür nicht. Dort dürfen andere nichts, das hast du in Teil 1 gesehen.

Gemeinsame Daten auf einem Server liegen oft in `/srv`. Das kennst du von Tag 3. Der Plan:

1. Einen Ordner `/srv/verkauf` anlegen.
2. Ihn der Gruppe `verkauf` geben.
3. Die Rechte so setzen: Die Gruppe darf alles, alle anderen nichts.

Für Schritt 2 brauchst du einen neuen Befehl: `chown`.

Tipp: Gibt es Javier oder die Gruppe `verkauf` bei dir nicht? Dann mach zuerst Schritt 1 und 2 der Aufgabe „Ein Ordner für den Verkauf“.

chown: den Besitzer ändern

`chown` kommt von „change owner“. Es ändert den Besitzer, die Gruppe oder beides.

```
cd ~/linux-kurs/tag08
touch bericht.txt
sudo chown javier bericht.txt
ls -l bericht.txt
# -rw-rw-r-- 1 javier carlos 0 Sep 30 11:05 bericht.txt
sudo chown $USER:verkauf bericht.txt
ls -l bericht.txt
# -rw-rw-r-- 1 carlos verkauf 0 Sep 30 11:05 bericht.txt
```

`$USER` kennst du von Tag 4. Es steht für deinen Namen.

- `sudo chown javier datei` ändert nur den Besitzer.
- `sudo chown javier:verkauf datei` ändert beides. Vor dem Doppelpunkt steht der Besitzer, dahinter die Gruppe.
- `sudo chgrp verkauf datei` ändert nur die Gruppe.

- `-R` ändert auch alles, was im Ordner liegt: `sudo chown -R javier:verkauf ordner`

Wozu sudo ? Den Besitzer darf nur root ändern. So kann niemand seine Dateien einfach einem anderen geben. Ohne `sudo` kommt `operation not permitted`.

root darf alles

Für root gelten die Rechte nicht. Darum geht mit `sudo`, was sonst verboten ist:

```
cat /etc/shadow
# cat: /etc/shadow: Permission denied
sudo cat /etc/shadow
```

Mit `sudo` siehst du die ganze Datei. Darin stehen die Passwörter aller Benutzer, verschlüsselt.

`sudo` darf nur benutzen, wer in der Gruppe `sudo` ist. Das hast du gestern bei `id` gesehen. Javier ist nicht in dieser Gruppe. Darum kam bei ihm `javier is not in the sudoers file`.

Schreib `sudo` aber nicht vor jeden Befehl. Was du mit `sudo` anlegst, gehört root. Danach kannst du es ohne `sudo` nicht mehr ändern:

```
sudo touch root-datei.txt
ls -l root-datei.txt
# -rw-r--r-- 1 root root 0 Sep 30 11:10 root-datei.txt
sudo rm root-datei.txt
```

In deinem Home-Ordner brauchst du nie `sudo`.

Der Ordner für den Verkauf

```
sudo mkdir /srv/verkauf
sudo chown $USER:verkauf /srv/verkauf
sudo chmod 770 /srv/verkauf
ls -ld /srv/verkauf
# drwxrwx--- 2 carlos verkauf 4096 Sep 30 11:15 /srv/verkauf
```

- `sudo mkdir`: `/srv` gehört root. Ohne `sudo` kommt `Permission denied`.
- `$USER:verkauf`: Du wirst Besitzer, die Gruppe ist `verkauf`.

- 770 : Du und die Gruppe dürft alles. Alle anderen dürfen nichts.

Javier ist in der Gruppe `verkauf` . Darum darf er im Ordner arbeiten:

```
su - javier
cd /srv/verkauf
echo "Angebot" > anbot.txt
ls -l
# total 4
# -rw-rw-r-- 1 javier javier 8 Sep 30 11:16 anbot.txt
exit
```

Wer nicht in der Gruppe ist, kommt nicht hinein. Er bekommt `Permission denied` .

Permission denied: den Grund finden

Diese Meldung heißt nur: Dir fehlt ein Recht. Mit drei Fragen findest du heraus, welches.

1. **Wer bin ich?** `whoami` und `id` . Bei `id` siehst du auch deine Gruppen.
2. **Wem gehört es?** `ls -l datei` oder `ls -ld ordner` .
3. **Welcher Block gilt für mich?** Besitzer, Gruppe oder andere. Fehlt dort `r` , `w` oder `x` ?

Ein Beispiel. Pablo will in den Ordner:

```
pablo@ubuntu:~$ cd /srv/verkauf
-bash: cd: /srv/verkauf: Permission denied
pablo@ubuntu:~$ id
uid=1003(pablo) gid=1004(pablo) groups=1004(pablo)
pablo@ubuntu:~$ ls -ld /srv/verkauf
drwxrwx--- 2 carlos verkauf 4096 Sep 30 11:15 /srv/verkauf
```

Pablo ist nicht carlos. Er ist auch nicht in `verkauf` . Also gilt für ihn der dritte Block: `---` . Er darf nichts.

Was ist die Lösung? Arbeitet Pablo im Verkauf? Dann kommt er in die Gruppe: `sudo usermod -aG verkauf pablo` . Arbeitet er woanders? Dann soll er nicht hinein. Der Fehler schützt den Ordner.

Achtung: Eine neue Gruppe gilt erst nach einer neuen Anmeldung. Ist Pablo schon angemeldet, tippt er `exit` . Dann meldet er sich neu an mit `su - pablo` .

Häufige Fehler

- **Operation not permitted bei chown.** sudo fehlt.
- **chown: invalid group: 'carlos:verkaf'** . Der Gruppenname ist falsch geschrieben, oder die Gruppe gibt es nicht. Prüfe mit `grep verkauf /etc/group` .
- **chown: cannot access 'verkauf': No such file or directory** . Du hast ein Leerzeichen statt des Doppelpunkts getippt: `chown carlos verkauf datei` . Richtig ist `chown carlos:verkauf datei` .
- **mkdir: cannot create directory '/srv/verkauf': Permission denied** . sudo fehlt. /srv gehört root.
- **Die neue Gruppe wirkt nicht.** Melde dich neu an: `exit` , dann wieder `su - javier` .
- **Eine Datei in deinem Home-Ordner gehört root.** Du hast sie mit `sudo` angelegt. Hol sie dir zurück: `sudo chown $USER:$USER datei`

Aufgaben · Teil 2

Ein Ordner für den Verkauf

Wozu? Im Verkauf sollen alle dieselben Dateien öffnen. Du baust ihnen einen Ordner, in den nur die Gruppe darf. So etwas kommt auch in der Klausur vor.

1. Prüfe Javier. Tippe im Terminal: `id javier`. Hinten muss `verkauf` stehen.
2. Steht dort `no such user`, oder fehlt `verkauf`? Dann tippe diese 4 Zeilen. Sonst mach weiter mit Schritt 3.

```
sudo groupadd verkauf
sudo useradd -m -s /bin/bash javier
echo "javier:Start123" | sudo chpasswd
sudo usermod -aG verkauf javier
```

Meldungen mit `already exists` sind hier kein Problem.

3. Lege den Ordner an: `sudo mkdir /srv/verkauf`. Kommt `File exists`, gibt es ihn schon. Dann mach weiter mit Schritt 4.
4. Gib ihn dir und der Gruppe: `sudo chown $USER:verkauf /srv/verkauf`
5. Setze die Rechte: `sudo chmod 770 /srv/verkauf`
6. Prüfe: `ls -ld /srv/verkauf`
7. Werde Javier: `su - javier`. Tippe das Passwort `Start123`.
8. Geh in den Ordner: `cd /srv/verkauf`
9. Lege eine Datei an: `echo "Angebot" > anbot.txt`
10. Prüfe: `ls -l`
11. Werde wieder du selbst: `exit`

So sehen Schritt 6 und Schritt 10 aus:

```
drwxrwx--- 2 carlos verkauf 4096 Sep 30 11:15 /srv/verkauf
total 4
-rw-rw-r-- 1 javier javier 8 Sep 30 11:16 anbot.txt
```

✓ Bestätigt

Pablo darf nicht hinein

Wozu? `Permission denied` wirst du oft sehen. Hier findest du den Grund mit den drei Fragen aus dem Text. Dann löst du das Problem.

Mach vorher die Aufgabe „Ein Ordner für den Verkauf“.

1. Lege Pablo an. Tippe im Terminal: `sudo useradd -m -s /bin/bash pablo`
2. Setze sein Passwort: `echo "pablo:Start123" | sudo chpasswd`
3. Werde Pablo: `su - pablo`. Tippe das Passwort `Start123`.
4. Versuche hineinzugehen: `cd /srv/verkauf`. Es kommt `Permission denied`.
5. Frage 1, wer bist du? Tippe `id`. Pablo ist nur in der Gruppe `pablo`.
6. Frage 2, wem gehört der Ordner? Tippe `ls -ld /srv/verkauf`.
7. Frage 3, welcher Block gilt für Pablo? Er ist nicht der Besitzer. Er ist nicht in `verkauf`. Also gilt der dritte Block: `---`.
8. Werde wieder du selbst: `exit`
9. Nimm Pablo in die Gruppe auf: `sudo usermod -aG verkauf pablo`
10. Melde Pablo neu an: `su - pablo`. Tippe das Passwort `Start123`. Nur so gilt die neue Gruppe.
11. Versuche es noch einmal: `cd /srv/verkauf`. Jetzt klappt es.
12. Zeige die Dateien: `ls -l`. Dort liegt `angebot.txt` von Javier.
13. Werde wieder du selbst: `exit`

So sehen Schritt 4 bis 6 aus. Die Nummern sind bei dir anders:

```
-bash: cd: /srv/verkauf: Permission denied
uid=1003(pablo) gid=1004(pablo) groups=1004(pablo)
drwxrwx--- 2 carlos verkauf 4096 Sep 30 11:15 /srv/verkauf
```

✓ Bestätigt

Ein Skript wie in der Klausur

Wozu? In der Klausur legst du Gruppen und Ordner mit einem Skript an. Dieses Skript baut einen zweiten Ordner, diesmal für das Lager.

Mach vorher die Aufgabe „Ein Ordner für den Verkauf“. Dann gibt es Javier.

1. Wechsle in den Ordner von heute: `cd ~/linux-kurs/tag08`
2. Öffne eine neue Datei: `nano lager.sh`
3. Tippe diese 2 Zeilen:

```
#!/bin/bash
# Legt die Gruppe lager und den Ordner /srv/lager an
```

4. Tippe darunter Schritt 1 und 2. Sie legen die Gruppe an und nehmen Javier auf:

```
echo "Schritt 1: Gruppe lager anlegen"
groupadd lager
sleep 2
echo "Schritt 2: javier in die Gruppe lager aufnehmen"
usermod -aG lager javier
sleep 2
```

5. Tippe darunter Schritt 3 und 4. Sie legen den Ordner an und setzen Besitzer und Gruppe:

```
echo "Schritt 3: Ordner /srv/lager anlegen"
mkdir /srv/lager
sleep 2
echo "Schritt 4: Besitzer und Gruppe setzen"
chown javier:lager /srv/lager
sleep 2
```

6. Tippe darunter Schritt 5 und 6. Sie setzen die Rechte und zeigen das Ergebnis:

```
echo "Schritt 5: Rechte setzen"
chmod 770 /srv/lager
sleep 2
```

```
echo "Schritt 6: Ergebnis zeigen"  
ls -ld /srv/lager
```

7. Speichere mit Strg+O und Enter. Beende mit Strg+X.
8. Mache das Skript ausführbar: `chmod +x lager.sh`
9. Starte es mit `sudo : sudo ./lager.sh`. Im Skript selbst steht kein `sudo`.

So sieht die Ausgabe aus:

```
Schritt 1: Gruppe lager anlegen  
Schritt 2: javier in die Gruppe lager aufnehmen  
Schritt 3: Ordner /srv/lager anlegen  
Schritt 4: Besitzer und Gruppe setzen  
Schritt 5: Rechte setzen  
Schritt 6: Ergebnis zeigen  
drwxrwx--- 2 javier lager 4096 Sep 30 11:40 /srv/lager
```

✓ Bestätigt

Quiz · Teil 2

Das Quiz gibt keine Punkte. Du kannst es so oft wiederholen, wie du willst.

Quiz abgegeben. Ergebnis: **5 / 5 richtig**

1. Welcher Befehl macht carlos zum Besitzer und verkauf zur Gruppe von bericht.txt ?

```
sudo chmod carlos:verkauf bericht.txt
```

```
sudo chown verkauf:carlos bericht.txt
```

```
sudo chown carlos:verkauf bericht.txt (Deine Antwort)
```

```
sudo chown carlos verkauf bericht.txt
```

Richtig

Erklärung:

chown ändert den Besitzer. Vor dem Doppelpunkt steht der Besitzer, dahinter die Gruppe. Mit Leerzeichen hält chown verkauf für eine Datei.

2. ls -ld zeigt: drwxrwx--- 2 carlos verkauf 4096 Sep 30 11:15 /srv/verkauf . Pablo ist nicht in verkauf . Was passiert bei cd /srv/verkauf ?

Er kommt hinein, darf aber nichts ändern.

Er kommt hinein und darf alles.

```
Permission denied . (Deine Antwort)
```

Er darf nur ls benutzen.

Richtig

Erklärung:

Pablo ist nicht der Besitzer und nicht in der Gruppe. Für ihn gilt der dritte Block --- . Ohne x kommt er nicht hinein.

3. Welche Zahl gibt dem Besitzer und der Gruppe alle Rechte und allen anderen keine?

Deine Antwort: 770

Richtig

Erklärung:

rwX ist 7, --- ist 0. Besitzer 7, Gruppe 7, andere 0: 770 .

4. Pablo ist angemeldet. Du tippst `sudo usermod -aG verkauf pablo` . Pablo bekommt bei `cd /srv/verkauf` trotzdem `Permission denied` . Warum?

Die neue Gruppe gilt erst nach einer neuen Anmeldung. (Deine Antwort)

Das `-a` war falsch.

Der Ordner braucht `777` .

Pablo muss `sudo cd` tippen.

Richtig

Erklärung:

Linux liest die Gruppen beim Anmelden. Pablo tippt `exit` und meldet sich mit `su - pablo` neu an. Dann klappt es.

5. Du tippst `chown javier liste.txt` ohne `sudo`. Es kommt `Operation not permitted`. Warum?

`liste.txt` hat kein `x`.

Nur root darf den Besitzer ändern. (Deine Antwort)

Javier hat kein Passwort.

Die Datei gehört schon javier.

Richtig

Erklärung:

Den Besitzer darf nur root ändern. Darum brauchst du `sudo chown`.

Bonusaufgaben · Teil 2

Freiwillig, für alle, die früher fertig sind. Bonusaufgaben zählen nicht für die Tagesmedaille. Sind alle Bonusaufgaben eines Tages bestätigt, gibt es eine Bonusmedaille.

Ein Skript, das fragt Bonus

Wozu? `lager.sh` baut nur den Ordner für das Lager. Ein Skript, das nach dem Namen fragt, baut dir einen Ordner für jede Gruppe.

Diesmal schreibst du das Skript selbst. Hier steht, was es tun soll. Die Zeilen findest du selbst.

Das soll dein Skript `ordner.sh` tun:

1. Es fragt nach dem Namen der Gruppe. Die Antwort kommt in die Variable `GRUPPE`.
2. Es legt die Gruppe an.
3. Es legt in `/srv` einen Ordner mit dem Namen der Gruppe an. Für die Gruppe `technik` also `/srv/technik`.
4. Der Besitzer ist `root`. Die Gruppe ist die neue Gruppe.
5. Die Gruppe darf alles. Alle anderen dürfen nichts.
6. Es zeigt das Ergebnis mit `ls -ld`.

Vor jeden Schritt kommt ein `echo`, zwischen zwei Schritte ein `sleep 2`.

Hilfen:

- `lager.sh` aus der Aufgabe „Ein Skript wie in der Klausur“ ist deine Vorlage.
- `read -p` kennst du von Tag 4.
- Eine Variable darf mitten in einem Pfad stehen: `/srv/$GRUPPE`
- Schreib kein `sudo` ins Skript. Du startest es mit `sudo ./ordner.sh`.

So testest du:

1. Speichere das Skript in `~/linux-kurs/tag08`. Mache es ausführbar.
2. Starte es: `sudo ./ordner.sh`. Tippe `technik`.
3. Starte es noch einmal. Tippe `marketing`.
4. Prüfe beide Ordner: `ls -l /srv`
5. Räume auf: Lösche beide Ordner und beide Gruppen, mit `sudo`. Die Befehle findest du selbst.

So kann der erste Start aussehen. Deine Texte bei `echo` dürfen anders sein:

Name der Gruppe: technik

Schritt 1: Gruppe technik anlegen

Schritt 2: Ordner /srv/technik anlegen

Schritt 3: Besitzer und Gruppe setzen

Schritt 4: Rechte setzen

Schritt 5: Ergebnis zeigen

```
drwxrwx--- 2 root technik 4096 Sep 30 12:10 /srv/technik
```

✓ Bestätigt

Eine Datei für die ganze Gruppe Bonus

Wozu? Ein gemeinsamer Ordner hat noch ein Problem. Hier findest du es selbst. Dann löst du es mit einem neuen Recht.

Mach vorher die Aufgaben „Ein Ordner für den Verkauf“ und „Pablo darf nicht hinein“. Dann sind Javier und Pablo in `verkauf`. In `/srv/verkauf` liegt `angebot.txt` von Javier.

1. Werde Pablo. Hänge eine Zeile an Javiers Datei an: `echo "Neuer Preis" >> /srv/verkauf/angebot.txt`
2. Es kommt `Permission denied`. Dabei ist Pablo doch in der Gruppe!
3. Finde den Grund mit den drei Fragen aus dem Text. Tipp: Schau genau, welche Gruppe `angebot.txt` hat. Werde danach wieder du selbst.
4. Jetzt kommt das neue Recht: `s` für die Gruppe eines Ordners. Tippe: `sudo chmod g+s /srv/verkauf`
5. Prüfe mit `ls -ld /srv/verkauf`. In der Mitte steht jetzt `rws` statt `rwX`. Ab jetzt bekommt jede neue Datei im Ordner die Gruppe des Ordners, also `verkauf`.
6. Teste es: Javier legt in `/srv/verkauf` eine neue Datei an. Welche Gruppe hat sie? Darf Pablo eine Zeile anhängen?
7. Für `angebot.txt` gilt das `s` nicht. Die Datei war schon vorher da. Gib ihr die richtige Gruppe. Den Befehl kennst du aus dem Abschnitt „chown: den Besitzer ändern“. Du brauchst `sudo`.
8. Teste Schritt 1 noch einmal.

So sieht Schritt 5 aus:

```
drwxrws--- 2 carlos verkauf 4096 Sep 30 11:50 /srv/verkauf
```

✓ Bestätigt

Das Geheimnis von /tmp Bonus

Wozu? In /tmp darf jeder Dateien anlegen. Trotzdem kann niemand die Dateien der anderen löschen. Das macht ein besonderes Recht. Du findest es und baust es dann selbst.

Mach vorher die Aufgabe „Pablo darf nicht hinein“. Dann gibt es Javier und Pablo.

1. Zeige die Rechte von /tmp: `ls -ld /tmp`. Hinten steht `rwt` statt `rwX`.
2. Werde Javier. Lege eine Datei `/tmp/javier.txt` an. Werde wieder du selbst.
3. Werde Pablo. Versuche, `/tmp/javier.txt` zu löschen. Was passiert? Fragt `rm` nach `write-protected`, tippe `y` und Enter. Werde wieder du selbst.
4. Normal gilt: Wer bei einem Ordner `w` hat, darf jede Datei darin löschen. Bei /tmp haben alle `w`. Das `t` ändert die Regel: Eine Datei darf nur ihr Besitzer löschen. Das `t` heißt **Sticky Bit**.
5. Baue jetzt selbst einen Ordner `/srv/austausch`. Der Besitzer ist `root`, die Gruppe ist `verkauf`. Die Gruppe darf alles, alle anderen nichts. Diese Befehle kennst du.
6. Setze das `t`: `sudo chmod +t /srv/austausch`
7. Prüfe mit `ls -ld /srv/austausch`. Hinten steht ein großes `T`. Das ist kein Fehler. Groß heißt: Die anderen haben kein `x`. Das `t` wirkt trotzdem.
8. Teste: Javier legt dort eine Datei an. Pablo versucht, sie zu löschen. Pablo legt eine eigene Datei an und löscht sie wieder.
9. Räume auf: `sudo rm -r /srv/austausch` und `sudo rm /tmp/javier.txt`

So sehen Schritt 1 und Schritt 7 aus:

```
drwxrwxrwt 12 root root 4096 Sep 30 12:30 /tmp
drwxrwx--T 2 root verkauf 4096 Sep 30 12:40 /srv/austausch
```

✓ Bestätigt

Aufräumen mit einem Skript Bonus

Wozu? Nach dem Üben willst du den alten Stand zurück. Ein Skript räumt alles auf einmal weg. Dann kannst du „Pablo darf nicht hinein“ und „Ein Skript wie in der Klausur“ noch einmal üben.

Mach vorher „Pablo darf nicht hinein“ und „Ein Skript wie in der Klausur“. Mach diese Aufgabe als letzte. Danach gibt es Pablo nicht mehr.

Schreib das Skript selbst. Hier steht, was es tun soll.

Das soll dein Skript `aufraeumen.sh` tun:

1. Es löscht den Ordner `/srv/lager` mit allem darin.
2. Es löscht die Gruppe `lager`.
3. Es löscht den Benutzer `pablo` mit seinem Home-Ordner.
4. Es zeigt mit `ls /srv`, was noch da ist.

Vor jeden Schritt kommt ein `echo`, zwischen zwei Schritte ein `sleep 2`.

Hilfen:

- `rm -r` kennst du von Tag 2.
- `userdel -r` und `groupdel` kennst du von Tag 7.
- Schreib kein `sudo` ins Skript. Du startest es mit `sudo ./aufraeumen.sh`.
- `/srv/verkauf`, `javier` und `verkauf` bleiben. Die brauchst du vielleicht noch.

So testest du:

1. Speichere das Skript in `~/linux-kurs/tag08`. Mache es ausführbar.
2. Starte es: `sudo ./aufraeumen.sh`
3. Prüfe: `id pablo` meldet `no such user`.

Kommt `user pablo is currently used by process`, ist noch ein Terminal als `pablo` offen. Tippe dort `exit`.

So kann die Ausgabe aussehen:

```
Schritt 1: Ordner /srv/lager loeschen
Schritt 2: Gruppe lager loeschen
Schritt 3: pablo loeschen
```

```
userdel: pablo mail spool (/var/mail/pablo) not found
```

Schritt 4: Prüfen

verkauf

✓ **Bestätigt**